

目 录

Contents

小组信息	1
实验环境	3
程序设计思路	4
整体架构	4
CFG 化简流水线	5
PDA 到 CFG 转换思路	5
核心算法	6
epsilon 产生式消除	6
单产生式消除	6
无用符号消除	7
PDA 到 CFG 的等价构造	8
输入格式与输出格式	8
CFG 输入格式	8
PDA 输入格式	10
输出格式	11
测试用例与执行效果	11
CFG 指定样例	11
PDA 指定样例	12
自动化测试	13
改进思路	14

小组信息

姓名	学号	角色	分工与负责模块
张恒基	2024210926	组长	CFG 数据结构与化简算法; cfg.py 、 cfg_parser.py 、 cfg_simplifier.py ，实现 epsilon 产生式消除、单产生式消除、无用符号消除，并整理 CFG 算法说明。
林旭东	2024210915	PDA / 转换	PDA 数据结构与 PDA→CFG 转换; pda.py 、 pda_parser.py 、 pda_to_cfg.py ，实现 PDA 输入解析、[p,A,q] 变量构造和转换算法说明。
尹浩铭	2024210910	入口 / 测试	命令行入口与测试验证; main.py 、 examples/ 、 tests/ ，整理实验指定样例，编写并运行单元测试，记录运行输出。
赵博宇	2024210908	文档 / 演示	实验报告与演示材料; README.md 、 docs/report.md 、 docs/executable.md 、 docs/video_script.md ，补充实验环境、输入输出格式、截图和演示视频脚本。

Table 1: 小组成员与分工（班级 2024211301，组长张恒基）

实验环境

项目	配置
操作系统	Windows 11
编程语言	Python 3.12
依赖情况	程序仅使用 Python 标准库（ dataclasses 、 itertools 、 argparse 、 re 、 unittest 等），无需安装第三方包；测试使用 unittest
运行入口	main.py ；可执行包 dist/formal_lang_lab2.exe
打包工具	PyInstaller（ --onefile ）

Table 2: 实验软硬件与运行环境

项目结构：

```
formal-language-lab2/
├─ main.py           # 命令行入口
├─ cfg.py            # CFG 数据结构
├─ cfg_parser.py     # CFG 文本解析器
├─ cfg_simplifier.py # CFG 化简算法
├─ pda.py            # PDA 数据结构
├─ pda_parser.py     # PDA 文本解析器
├─ pda_to_cfg.py     # PDA 到 CFG 转换算法
├─ examples/
│   ├─ grammar_sample.txt # 指定 CFG 样例（块格式）
│   ├─ specified_cfg.txt  # 指定 CFG 样例（符号格式）
│   ├─ pda_sample.txt    # 指定 PDA 样例（块格式）
│   └─ specified_pda.txt  # 指定 PDA 样例（数学符号格式）
├─ tests/
│   ├─ test_cfg_parser.py
│   └─ test_cfg_simplifier.py
```

```

├── test_pda_parser.py
├── test_pda_to_cfg.py
├── test_main.py
└── docs/
    ├── report.md          # 本实验报告 (Markdown 版)
    ├── requirements.md    # 实验要求
    ├── executable.md      # 可执行程序说明
    └── video_script.md    # 演示视频脚本

```

程序设计思路

整体架构

项目采用模块化分层设计，按功能拆分为三层：数据结构层（

`cfg.py`

、

`pda.py`

）、解析层（

`cfg_parser.py`

、

`pda_parser.py`

）、算法层（

`cfg_simplifier.py`

、

`pda_to_cfg.py`

），最外层由

`main.py`

统一调度。

这种分层的优势在于：

- 算法与数据解耦：化简算法只操作

CFG

对象，不关心输入文本格式；转换算法只操作

PDA

对象，不关心输出格式。

- 可组合性：PDA 转 CFG 后得到的

CFG

对象可直接传入化简流水线，实现实验要求的“先转换再化简”的串联流程。

- 可测试性：每个模块可独立编写单元测试，不依赖命令行或文件 I/O。

源文件	职责
cfg.py	CFG 产生式与变量数据结构
cfg_parser.py	CFG 文本解析（含 eps 多别名词）
cfg_simplifier.py	三步化简算法（epsilon 消除、单产生式消除、无用符号消除）
pda.py	PDA 五元组与迁移结构
pda_parser.py	PDA 文本解析（支持数学符号格式与块格式）
pda_to_cfg.py	PDA→CFG 构造（标准 [p,A,q] 变量法）
main.py	子命令 simplify / pda2cfg / demo

Table 3: 模块划分

CFG 化简流水线

CFG 化简流程按固定顺序依次执行四个步骤：

输入 CFG

- (1) 消除 epsilon 产生式 (eliminate_epsilon Productions)
- (2) 消除单产生式 (eliminate_unit Productions)
- (3) 消除非生成符号 (remove_non_generating_symbols)
- (4) 消除不可达符号 (remove_unreachable_symbols)
- 输出等价的无 epsilon、无单产生式、无无用符号的 CFG

顺序不可调换的原因：消除 epsilon 产生式可能引入新的单产生式，因此必须先消 epsilon 再消单产生式；消除单产生式可能使某些变量变为非生成，因此消单产生式必须在消除非生成符号之前；消除非生成符号可能切断某些变量的可达路径，因此先消非生成再消不可达，确保只保留“既能生成终结串、又能被开始符号到达”的有用符号。

PDA 到 CFG 转换思路

采用教材标准构造：对于空栈接受的 PDA，引入形如 [p,A,q] 的变量，其语义为“从状态 p 出发，栈顶为 A ，经过若干步后弹出 A 并到达状态 q ，期间读入的输入串”。

开始符号 S 的产生式为 $S \rightarrow [q_0, z_0, q]$ （对所有状态 q ），表示从初始状态 q_0 出发，初始栈符号 z_0 在栈顶，最终弹出 z_0 到达某个状态 q ，此时栈为空，PDA 接受。

迁移的处理分两种情况：

- 弹出（压入为空）： $\delta(p, a, A) = \{(q, \varepsilon)\}$ ，则添加产生式 $[p, A, q] \rightarrow a$ 。
- 压入 k 个符号： $\delta(p, a, A) = \{(q, B_1 B_2 \dots B_k)\}$ ，则对任意可能的状态序列 $r_1, r_2, \dots, r_{k-1}, q_k$ ，枚举中间状态，添加产生式 $[p, A, q_k] \rightarrow a [q, B_1, r_1] [r_1, B_2, r_2] \dots [r_{k-1}, B_k, q_k]$ 。

核心算法

epsilon 产生式消除

问题描述：形如 $A \rightarrow \varepsilon$ 的产生式称为 epsilon 产生式。消除 epsilon 产生式的目标是得到一个不含 ε 产生式的等价文法（若原语言不含空串 ε ）。

算法步骤：

- 步骤一：计算 **nullable** 变量集合（不动点迭代）
 - 初始化 nullable = $\emptyset$ 。
 - 重复扫描所有产生式：若存在产生式 $A \rightarrow \alpha$ ，且 α 中所有符号都在 nullable 中（空串 α 视为自动满足），则将 A 加入 nullable。
 - 直到 nullable 不再增长。
- 步骤二：扩展产生式右部：对每个产生式 $A \rightarrow X_1 X_2 \dots X_n$
 - 找出右部中所有 nullable 符号的位置。
 - 枚举这些位置的所有删除组合（含删除 0 个，即保留原右部）。
 - 对每种组合，将删除后的右部作为新产生式加入结果文法。
 - 若删除后右部为空（即全部符号被删），不加入（除非是保留开始符号 ε 的可选模式）。
- 步骤三：删除原 **epsilon** 产生式：上述步骤中跳过空右部，自然实现了删除。

复杂度分析：nullable 计算最多迭代 $|V|$ 次，每次扫描 $|P|$ 个产生式，复杂度 $O(|V| \cdot |P|)$ 。扩展右部时，若右部有 k 个 nullable 符号，则枚举 2^k 种组合。最坏情况下 nullable 符号数量可达到 $|V|$ ，但实际文法中通常很小。

代码实现要点（

cfg_simplifier.py:19-39

)：使用

itertools.combinations

枚举所有删除组合，

find_nullable_variables()

返回 nullable 变量集合。

单产生式消除

问题描述：形如 $A \rightarrow B$ (A, B 均为变量) 的产生式称为单产生式。单产生式本身不产生终结符，但可能形成链 $A \Rightarrow B \Rightarrow C \Rightarrow \dots$ ，增加推导步数且使文法膨胀。

算法步骤：

- 步骤一：计算 **unit-closure**：对每个变量 A ，通过 BFS 计算其通过单产生式可达的所有变量集合 unit_closure[A]。
 - 初始化 unit_closure[A] = {A}。
 - 重复：若 $B \in \text{unit_closure}[A]$ 且存在单产生式 $B \rightarrow C$ ，则将 C 加入。
 - 直到不再增长。
- 步骤二：替换产生式：对每个变量 A ，遍历 unit_closure[A] 中的每个变量 B ：
 - 将 B 的所有非单产生式右部（长度 $\neq 1$ 或该符号不是变量）加入 A 的产生式集合。
 - 不再保留任何单产生式。

复杂度分析：unit-closure 计算对 $|V|$ 个变量各做一次 BFS，每次 BFS 遍历可能包含所有变量，复杂度 $O(|V| \cdot (|V| + |P|))$ 。替换阶段复杂度 $O(|V| \cdot |P|)$ 。

代码实现要点（

cfg_simplifier.py:58-74

):

_unit_reachable_variables()

使用显式栈的 DFS 实现传递闭包；

_is_unit_production()

检查右部长度是否为 1 且该符号是否属于变量集。

无用符号消除

无用符号分为两类：非生成符号（不能推导出任何终结字符串）和不可达符号（从开始符号出发无法到达）。消除必须按“先非生成、后不可达”的顺序进行。

第一步：消除非生成符号

算法步骤：

1. 计算生成变量集合（不动点迭代）：

- 初始化 $\text{generating} = \emptyset$ 。
- 重复：对每个产生式 $A \rightarrow \alpha$ ，若 α 中所有变量都在 generating 中（终结符自动视为可生成），则将 A 加入 generating 。
- 直到不再增长。

2. 过滤产生式：删除左部不在 generating 中的产生式；对于保留的产生式，若其右部包含非生成变量，则该产生式也被删除（因为非生成变量永远无法推导出终结字符串）。

第二步：消除不可达符号

算法步骤：

1. 计算可达变量集合（BFS）：

- 初始化 $\text{reachable} = \{S\}$ 。
- 重复：对每个可达变量 A ，检查其所有产生式右部，将其中出现的变量加入 reachable 。
- 直到不再增长。

2. 过滤产生式：只保留左部在 reachable 中的产生式；对于保留的产生式，若右部包含不可达变量，同样删除。

复杂度分析：两个步骤均为不动点迭代，复杂度各为 $O(|V| \cdot |P|)$ 。

代码实现要点（

cfg_simplifier.py:77-138

):

find_generating_variables()

和

find_reachable_variables()

分别实现上述迭代过程。注意在判定“右部符号是否满足条件”时，终结符总是自动满足。

PDA 到 CFG 的等价构造

前置条件：PDA 必须以空栈方式接受（

`accepts_by_empty_stack = True`

）。若 PDA 使用终态接受，需先转换为空栈接受。

算法步骤：

1. 步骤一：构造变量集合：对所有状态 $p, q \in Q$ 和栈符号 $A \in \Gamma$ ，构造变量 $[p, A, q]$ ，其语义为“从状态 p 出发，栈顶为 A ，读入某个串后弹出 A 并到达状态 q ”。变量总数 $= |Q|^2 \times |\Gamma|$ 。
2. 步骤二：构造开始产生式：对每个状态 $q \in Q$ ，添加：

$$S \rightarrow [q_0, z_0, q]$$

其中 q_0 为初始状态， z_0 为初始栈符号。

3. 步骤三：构造迁移产生式：对每个迁移 $\delta(p, a, A) = \{(q, B_1 B_2 \dots B_k)\}$ ：

- 若 $k = 0$ （仅弹出）：添加 $[p, A, q] \rightarrow a$ 。
- 若 $k \geq 1$ ：枚举所有中间状态 $r_1, r_2, \dots, r_{k-1} \in Q$ ，添加：

$$[p, A, q_k] \rightarrow a [q, B_1, r_1] [r_1, B_2, r_2] \dots [r_{k-1}, B_k, q_k]$$

其中 q_k 取遍所有状态。

4. 步骤四： ϵ 迁移处理：若输入符号为 ϵ （即 $\delta(p, \epsilon, A)$ ），则上述步骤中 $a = \epsilon$ ，产生式右部以变量串开头。

复杂度分析：构造的变量数为 $|Q|^2 \times |\Gamma|$ 。对于压入 k 个符号的迁移，需枚举 $|Q|^{k-1}$ 个中间状态组合，再乘以 $|Q|$ 种 q_k 选择，共 $|Q|^k$ 种。总产生式数量在最坏情况下为 $O(|Q|^{K+1} \times |\Gamma|)$ ，其中 K 为最大压入长度。

代码实现要点（

`pda_to_cfg.py:15-95`

）：使用

`itertools.product`

枚举中间状态；

`_add_transition_productions()`

分别处理 push 长度为 0 和 ≥ 1 的情况；对仅支持空栈接受的 PDA 做了显式校验。

输入格式与输出格式

CFG 输入格式

支持两种写法风格：

紧凑风格（单字符符号直接拼接）：

```
S -> a | bA | B | ccD
A -> abB | epsilon
B -> aA
```

C -> ddC
D -> ddd

空格分隔风格（多字符符号用空格隔开，如 $[q_0, B, q_1]$ ）：

S -> b $[q_0, B, q_1]$
 $[q_0, B, q_1]$ -> a | b $[q_0, B, q_1]$

解析规则：

- 产生式箭头支持

->

、

=>

、

-->

、

:

、

=

及 Unicode 箭头

→

◦

- ε 的别名：

epsilon

、

eps

、

e

、

lambda

、

empty

、 Unicode

ε

◦

- 用

|

分隔多个候选右部。

- #

至行尾为注释。

- 起始终结符从产生式自动推导。

PDA 输入格式

支持两种格式：

格式一：数学符号格式

```
M = ({q0,q1}, {a,b}, {B,z0}, delta, q0, z0, empty)
delta(q0,b,z0) = {(q0,Bz0)}
delta(q0,b,B) = {(q0,BB)}
delta(q0,a,B) = {(q1,epsilon)}
delta(q1,a,B) = {(q1,epsilon)}
delta(q1,epsilon,B) = {(q1,epsilon)}
delta(q1,epsilon,z0) = {(q1,epsilon)}
```

格式二：块格式（推荐）

```
states: q0 q1
input_symbols: a b
stack_symbols: B z0
start_state: q0
start_stack: z0
accept: empty_stack
transitions:
q0,b,z0 -> q0,B z0
q0,b,B -> q0,B B
q0,a,B -> q1,eps
q1,a,B -> q1,eps
q1,eps,B -> q1,eps
q1,eps,z0 -> q1,eps
```

解析规则：

- 自动识别 Unicode δ 、 ϵ 、 Φ 等字符并归一化。
- 全角字符（=，（）等）自动转为半角。
- accept

字段：

empty_stack

/

empty

/

phi

/

Φ

表示空栈接受。

- 栈符号串（如

Bz0

) 会根据已知栈符号集合自动分词。

输出格式

输出为标准产生式列表，按变量名排序（开始符号 S 始终排在最前）：

```
S -> a | aA | b | bA | ccD
A -> abB
B -> a | aA
D -> ddd
```

多字符符号自动以空格分隔输出， ε 统一显示为

epsilon

。

测试用例与执行效果

CFG 指定样例

输入文法（

examples/grammar_sample.txt

）：

```
S → a | bA | B | ccD
A → abB | ε
B → aA
C → ddC
D → ddd
```

分析：

- A 有 ε 产生式 $A \rightarrow \varepsilon$ ，因此 A 是 nullable。
- $B \rightarrow aA$ 中的 A 是 nullable，展开后得到 $B \rightarrow aA \mid a$ 。
- $S \rightarrow bA$ 展开后得到 $S \rightarrow bA \mid b$ ； $S \rightarrow B$ 为单产生式，unit-closure 将 B 的非单产生式（如 $B \rightarrow aA$ 及其 epsilon 展开）并入 S 。
- $C \rightarrow ddC$ ： C 只产生自己，属于非生成符号，应被删除。

运行命令：

```
py main.py simplify examples\grammar_sample.txt
```

输出：

```
S -> a | aA | b | bA | ccD
A -> abB
B -> a | aA
D -> ddd
```

结果分析：

- C 被正确删除（非生成变量， $C \rightarrow ddC$ 永远无法终止）。
- ε 产生式 $A \rightarrow \varepsilon$ 被消除，取而代之的是在包含 A 的右部中删除 A 的版本（如 $S \rightarrow aA \mid a$ ， $B \rightarrow aA \mid a$ ）。

- 单产生式 $S \rightarrow B$ 被消除； $B \rightarrow aA$ 不是单产生式，其经 nullable 展开后的右部通过 unit-closure 并入 S （如输出中的 $S \rightarrow a \mid aA$ 等）。
- $D \rightarrow ddd$ 保留（ D 可生成终结字符串且从 S 可达）。

```
PS C:\projects\formal-language-lab2> py main.py simplify examples\grammar_sample.txt
S -> a | aA | b | bA | ccD
A -> abB
B -> a | aA
D -> ddd
PS C:\projects\formal-language-lab2>
```

Figure 1: CFG 化简运行截图

PDA 指定样例

输入 PDA (

examples/pda_sample.txt

):

$M = (\{q_0, q_1\}, \{a, b\}, \{B, z_0\}, \delta, q_0, z_0, \Phi)$

$\delta(q_0, b, z_0) = \{(q_0, Bz_0)\}$	// 读 b, 栈顶 z_0 , 压入 Bz_0 (即压 B)
$\delta(q_0, b, B) = \{(q_0, BB)\}$	// 读 b, 栈顶 B, 压入 BB (即再压一个 B)
$\delta(q_0, a, B) = \{(q_1, \varepsilon)\}$	// 读 a, 栈顶 B, 弹出
$\delta(q_1, a, B) = \{(q_1, \varepsilon)\}$	// 读 a, 栈顶 B, 弹出
$\delta(q_1, \varepsilon, B) = \{(q_1, \varepsilon)\}$	// 空输入, 栈顶 B, 弹出
$\delta(q_1, \varepsilon, z_0) = \{(q_1, \varepsilon)\}$	// 空输入, 栈顶 z_0 , 弹出 (清空栈)

语言识别：该 PDA 识别形如「先若干 b 、再若干 a 」的串，且 a 的个数不能超过 b 的个数（每读一个 b 在栈上压入一个 B ， a 与 ε 迁移用于弹出 B ）。用集合写法可写为

$$L = \{b^m a^n \mid m \geq 1, n \geq 1, n \leq m\}$$

（与单元测试 `test_simplified_cfg_matches_specified_pda_examples` 一致；不是 $b^n a^n$ ）。在 q_0 每读 b 压栈，首次读 a 转入 q_1 ，最后用 ε 清空 z_0 完成空栈接受。

运行命令：

```
py main.py pda2cfg examples\pda_sample.txt --simplify
```

输出：

```
S -> b [q0,B,q1]
[q0,B,q1] -> a | b [q0,B,q1] | b [q0,B,q1] [q1,B,q1]
[q1,B,q1] -> a
```

结果分析：

- 开始符号 S 只推出一个变量 $[q_0, B, q_1]$ （从 q_0 出发弹出 B 到 q_1 的变量被保留），说明从 q_0 出发弹出初始栈符号 z_0 后只能到达 q_1 。
- $[q_0, B, q_1] \rightarrow a$ 对应 $\delta(q_0, a, B) = \{(q_1, \varepsilon)\}$ ：读 a 弹出 B 。
- $[q_0, B, q_1] \rightarrow b [q_0, B, q_1]$ 对应 $\delta(q_0, b, B) = \{(q_0, BB)\}$ ，中间状态取 $r = q_1$ ，第二个 B 弹出的终点为 q_1 。

- $[q_0, B, q_1] \rightarrow b [q_0, B, q_1] [q_1, B, q_1]$ 对应上述迁移中中间状态取 $r = q_0$ 的情况，形成递归产生式。
- $[q_1, B, q_1] \rightarrow a$ 对应 $\delta(q_1, a, B) = \{(q_1, \varepsilon)\}$ 。
- 化简后产生式由原始的（转换后）数十条降至 4 条，消除了大量冗余变量（如与 q_0 到达 q_0 相关的变量在化简过程中被判定为非生成或不可达而被删除）。

语义验证：化简后文法可生成 ba 、 bba 、 $bbaa$ 、 $bbbaaa$ 等 ($m \geq n \geq 1$)；不能生成空串、单独的 b 或 a 、以及 a 多于 b 的串（如 baa ）。这与上述 $L = \{b^m a^n \mid m, n \geq 1, n \leq m\}$ 一致。

```
PS C:\projects\formal-language-lab2> py main.py pda2cfg examples\pda_sample.txt --simplify
Simplified CFG:
S -> b [q0,B,q1]
[q0,B,q1] -> a | b [q0,B,q1] | b [q0,B,q1] [q1,B,q1]
[q1,B,q1] -> a
PS C:\projects\formal-language-lab2>
```

Figure 2: PDA 转 CFG 并化简运行截图

自动化测试

共编写 12 个单元测试，覆盖所有模块：

测试文件	测试数	覆盖内容
test_cfg_parser.py	2	指定 CFG 解析、多字符符号解析
test_cfg_simplifier.py	3	nullable 变量计算、指定 CFG 化简、单产生式环检测
test_pda_parser.py	2	块格式 PDA 解析、数学符号格式 PDA 解析
test_pda_to_cfg.py	3	标准变量构造、各压栈长度处理、化简后文法语义正确性
test_main.py	2	demo 命令输出、CLI 别名兼容性

Table 4: 单元测试覆盖一览

其中

test_pda_to_cfg.py

的语义测试尤为关键：它从化简后的文法执行 BFS 推导，枚举长度 $\leq N$ 的所有生成终结符串，然后逐串验证是否属于 PDA 识别的语言。这确保了“PDA 转换 $\rightarrow$ CFG 化简”整个流水线的语义正确性。

运行命令：

```
py -m unittest discover -s tests
```

运行结果：Ran 12 tests in 0.08s — OK

全部 12 项测试通过。

```
PS C:\projects\formal-language-lab2> C:\Users\bos\AppData\Local\Programs\Python\Python312\python.exe -m unittest discover -s tests
.....
-----
Ran 12 tests in 0.005s

OK
PS C:\projects\formal-language-lab2>
```

Figure 3: 单元测试运行截图

改进思路

- 图形化界面：当前程序为命令行界面，可增加基于 Web（如 Streamlit 或 Gradio）的图形界面，支持逐步展示化简过程（显示每一步前后的 CFG 变化），便于课堂演示和教学。
- 边界用例覆盖：增加更多边界用例的测试，如所有符号均为非生成的退化文法、仅含 ε 的文法、含复杂环的单产生式链、压栈长度超过 3 的 PDA 迁移等。
- 保留开始符号 ε 的可选模式：当原文法生成的语言包含空串时，完全消除 ε 产生式会使开始符号也不出现 ε ，虽然语言等价但推导树结构可能改变。可提供

`--keep-start-epsilon`

选项，当开始符号原本 nullable 时保留 $S \rightarrow \varepsilon$ 。

- 终态接受 **PDA** 支持：当前仅支持空栈接受，可扩展为也支持终态接受，通过自动插入从终态到空栈的 ε 迁移实现等价转换。
- 化简过程可视化输出：增加

`--verbose`

选项，输出每一步化简后的中间文法，帮助用户理解每一步消除的效果。